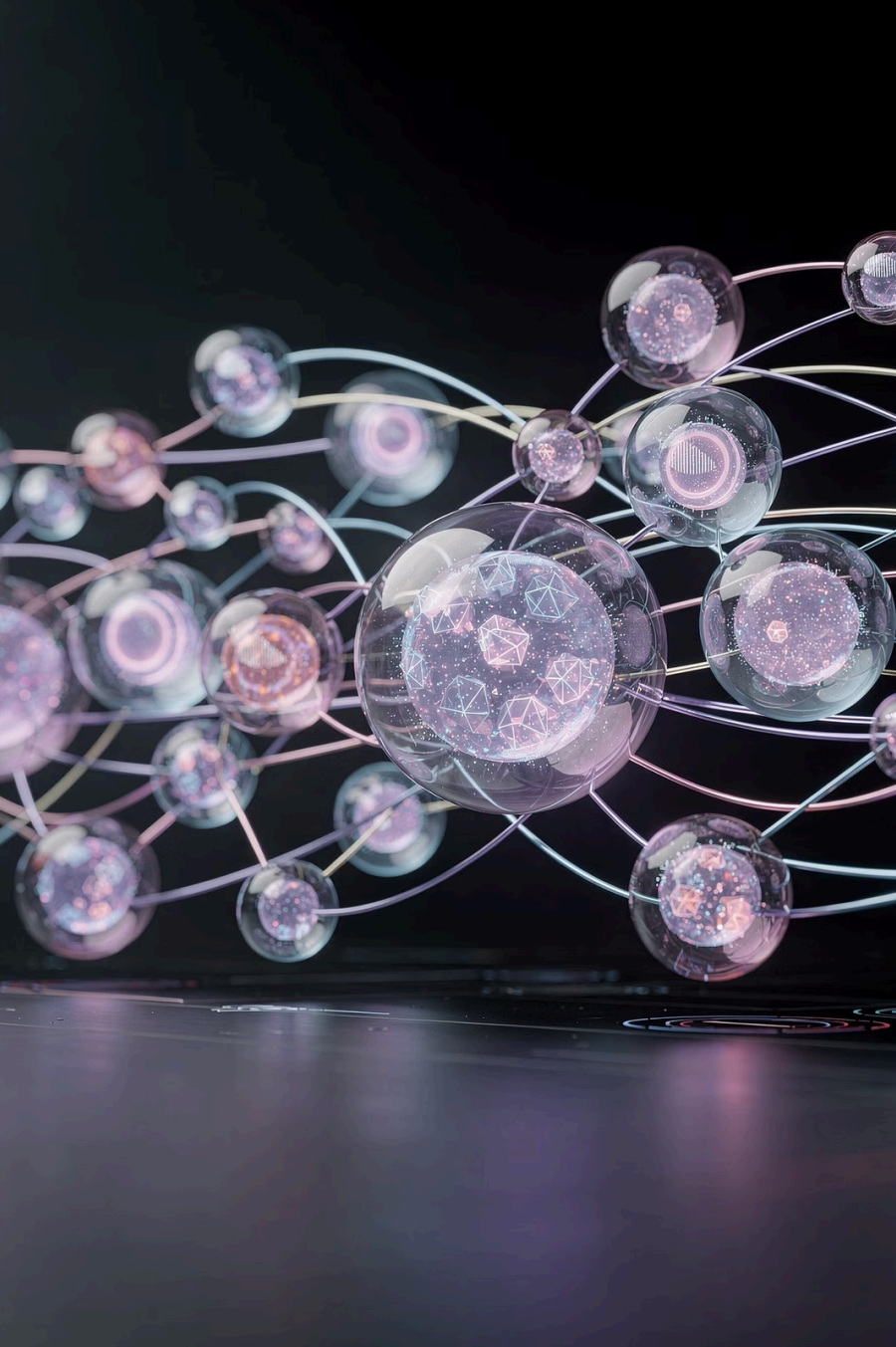




Estructuras de Datos Complejas en Python

Un viaje visual y progresivo para aprender a organizar, almacenar y manipular información de forma eficiente. Desde listas hasta diccionarios, explorando tuplas, sets y las bases para trabajar con estas estructuras en Python.

¿Por qué necesitamos estructuras de datos?

Imagina que tienes que gestionar la lista de alumnos de una clase, el turno en un cajero automático o los productos de un carrito de compra online. **Sin estructuras de datos, el caos está garantizado.**

El problema

Guardar cientos de datos sin organización hace que buscar, modificar o eliminar información sea lento, tedioso y propenso a errores.

La solución

Las estructuras de datos nos permiten **organizar la información de forma lógica**, adaptada a cada problema, para que nuestros programas sean eficientes y fáciles de mantener.



Tipo de Dato Abstracto (TDA)

Un **TDA** define *qué operaciones* podemos hacer con una estructura de datos, sin preocuparnos de *cómo están implementadas internamente*. Es como usar un cajero automático: sabemos que podemos ingresar dinero, retirarlo y consultar el saldo, pero no necesitamos saber la electrónica que hay detrás.



Lista

Colección ordenada de elementos. Permite acceso por índice.



Diccionario

Almacena pares clave→valor. Acceso rápido por clave única.

Listas Anidadas en Python

Una **lista anidada** es una lista que contiene otras listas como elementos. Son perfectas para representar **matrices, tablas o datos agrupados**. Muy útiles en procesamiento de imágenes, álgebra lineal y gestión de datos tabulares.

```
# Matriz 3x3 con listas anidadas
matriz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Acceder al elemento fila 1, columna 2
print(matriz[1][2]) # Resultado: 6

# Recorrer la matriz completa
for fila in matriz:
    for elemento in fila:
        print(elemento, end=" ")
    print()
```

  **¿Lo sabías?** Las listas anidadas son la base de bibliotecas como NumPy, que maneja matrices de millones de datos para ciencia e inteligencia artificial.

Casos de uso de listas anidadas



Tablero de ajedrez

Un tablero 8×8 se representa como una lista de 8 listas, cada una con 8 casillas. Ideal para juegos y simulaciones.

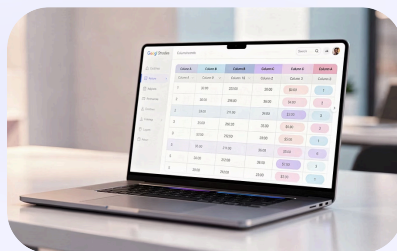
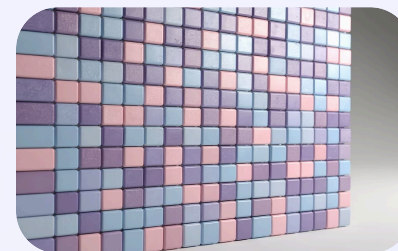


Tabla de datos

Almacena registros de alumnos, productos o cualquier información tabular antes de usar bases de datos.



Procesamiento de imágenes

Una imagen digital es una matriz de píxeles. Cada píxel tiene valores RGB almacenados en listas anidadas.

? 🤔 **Pregunta para reflexionar:** ¿Qué otras situaciones de tu día a día podrían representarse con una tabla de filas y columnas?



Tuplas — Datos que no cambian

Una **tupla** es como una lista, pero **inmutable**: una vez creada, no se puede modificar. Son perfectas para representar datos que no deben cambiar, como coordenadas geográficas, colores RGB o registros de base de datos.

Cuándo usar tuplas

- Coordenadas (latitud, longitud)
- Colores RGB (255, 128, 0)
- Fechas fijas (año, mes, día)
- Claves en diccionarios

Desempaquetado

```
# Crear una tupla  
punto = (40.4168, -3.7038)
```

```
# Desempaquetar  
lat, lon = punto  
print(f"Latitud: {lat}")  
print(f"Longitud: {lon}")
```

```
# Tupla como retorno múltiple  
def minmax(lista):  
    return min(lista), max(lista)
```

Sets — Conjuntos sin duplicados

Un **set** (conjunto) es una colección **no ordenada y sin elementos duplicados**. Son extremadamente útiles para eliminar valores repetidos y realizar operaciones matemáticas de conjuntos de forma eficiente.

```
# Crear un set
frutas = {"manzana", "pera", "naranja", "manzana"}
print(frutas) # {'manzana', 'pera', 'naranja'}
```

```
# Operaciones de conjuntos
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
```

```
print(a | b) # Unión: {1,2,3,4,5,6}
print(a & b) # Intersección: {3,4}
print(a - b) # Diferencia: {1,2}
```

Unión (|)

Todos los elementos de ambos conjuntos, sin repetir.

Intersección (&)

Solo los elementos comunes a los dos conjuntos.

Diferencia (-)

Elementos en A que no están en B.

Diccionarios — El poder de clave→valor

Un **diccionario** almacena información como una agenda: cada entrada tiene un **nombre (clave)** y una **información asociada (valor)**. Es la estructura más versátil de Python para modelar objetos del mundo real.

+ Agregar

```
d["clave"] = valor
```

✎ Modificar

```
d["clave"] = nuevo_valor
```

🗑 Eliminar

```
del d["clave"]
```

↻ Recorrer

```
for k, v in d.items()
```


Diccionarios en acción

Ejemplo práctico: ficha de producto en tienda online

```
producto = {  
    "nombre": "Auriculares Bluetooth",  
    "precio": 49.99,  
    "stock": 120,  
    "categoría": "Electrónica"  
}
```

Acceder a un valor

```
print(producto["nombre"])    # Auriculares Bluetooth
```

Modificar stock tras una venta

```
producto["stock"] -= 1
```

Recorrer todos los campos

```
for clave, valor in producto.items():  
    print(f"{clave}: {valor}")
```

Eliminar un campo

```
del producto["categoría"]
```



Pregunta: ¿Podrías crear un diccionario para representar un alumno con su nombre, nota y asignatura? ¿Qué claves usarías?

¿Cuándo usar cada estructura?

Elegir la estructura de datos correcta es una de las habilidades más importantes de un programador. Esta guía visual te ayudará a tomar la decisión adecuada según el contexto de tu problema.

1

 Lista – Necesitas orden y acceso por índice. Los datos cambian.

2

 Tupla – Datos fijos e inmutables.

3

 Set – Unicidad.
Operaciones de conjuntos.

4

 Diccionario – Asociar claves con valores.

Recorriendo estructuras con for

El bucle `for` de Python es una herramienta poderosa y flexible que se adapta a la naturaleza de cada estructura de datos. Entender cómo iterar correctamente es fundamental para procesar y manipular la información almacenada en listas, tuplas, sets y diccionarios.

```
# 1. Lista
mi_lista = ["manzana", "pera", "uva"]

for elemento in mi_lista:
    print(elemento) # manzana, pera, uva

for i, elemento in enumerate(mi_lista):
    print(f"Índice {i}: {elemento}") # Índice 0: manzana, etc.

# 2. Tupla (igual que lista, también desempquetado)
mi_tupla = ("Juan", 30, "Ingeniero")
puntos = [(1,2), (3,4), (5,6)]

for dato in mi_tupla:
    print(dato) # Juan, 30, Ingeniero

for x, y in puntos:
    print(f"Punto: ({x}, {y})") # Punto: (1, 2), etc.

# 3. Set (sin orden garantizado)
mi_set = {"rojo", "verde", "azul"}

for color in mi_set:
    print(color) # (el orden puede variar)

# 4. Diccionario
mi_diccionario = {
    "nombre": "Alice",
    "edad": 25,
    "ciudad": "Nueva York"
}

for clave in mi_diccionario: # Itera sobre las claves
    print(clave) # nombre, edad, ciudad

for clave, valor in mi_diccionario.items(): # Itera sobre pares clave-valor
    print(f"{clave}: {valor}") # nombre: Alice, etc.

for clave in mi_diccionario.keys(): # Explícitamente sobre claves
    print(clave)

for valor in mi_diccionario.values(): # Explícitamente sobre valores
    print(valor)
```



Lista

```
for elemento in lista
for i, el in enumerate(lista)
```



Tupla

```
for elemento in tupla
for x, y in lista_de_tuplas
```



Set

```
for elemento in mi_set
(Orden no garantizado)
```



Diccionario

```
for clave, valor in d.items()
(O .keys(), .values())
```

  **Consejo:** Para diccionarios, `.items()` es el método más versátil, ya que te permite acceder a la clave y su valor asociado en cada iteración.

Comparativa final de estructuras

Estructura	Ordenada	Mutable	Duplicados	Acceso
Lista	✓ Sí	✓ Sí	✓ Sí	Por índice
Tupla	✓ Sí	✗ No	✓ Sí	Por índice
Set	✗ No	✓ Sí	✗ No	Pertenencia
Diccionario	✓ Sí (3.7+)	✓ Sí	✗ Claves no	Por clave

📌 📌 Recuerda: no existe una estructura «mejor que todas». La clave está en **elegir la herramienta correcta para cada problema.**

El camino recorrido hoy



Has recorrido las estructuras de datos más importantes de Python: desde listas anidadas hasta los diccionarios, pasando por tuplas y sets. Cada una tiene su lugar y su momento. Ahora tienes la caja de herramientas completa.